Melanomaly

mole recognizer

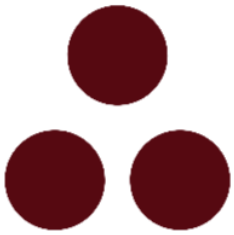


Fig. 1: icona dell'app

Introduzione:

La seguente applicazione ha come obiettivo quello di offrire all'utente un rapido controllo di una possibile anomalia dei nei del suo corpo. Tramite l'utilizzo della camera del telefono, l'utente acquisisce una foto del suo neo, di seguito tale foto verrà analizzata da una rete neurale che deciderà se l'immagine del neo ricevuta rappresenta un neo benigno o un neo maligno. A prescindere dall'esito della decisione l'utente può mandare al medico la foto analizzata tramite mail per ricevere il parere di un esperto a riguardo.

È doveroso ricordare che, prima di fare tutto ciò, l'utente accede alle funzionalità mediante registrazione di un account personale e successivo login cosicché l'app sia utilizzabile da più utenti ma che questi ultimi abbiano le loro foto (di tratti personali) separate da quelle degli altri. Le foto scattate e le informazioni dell'account creato sono salvate in un database relazionale che permette di associare all' utente tutte e sole le foto da lui scattate.

L'utente può comodamente accedere allo storico di tutte le (sue) foto tramite un recycler View che mostra tutte le foto passate e i relativi score dati dalla rete neurale.

Un vantaggio dell'app è l'analisi della foto in background tramite AsyncTask poiché questa risulta essere un'operazione gravosa che porterebbe inevitabilmente al blocco del funzionamento dell'app.

Schema concettuale dell'app:

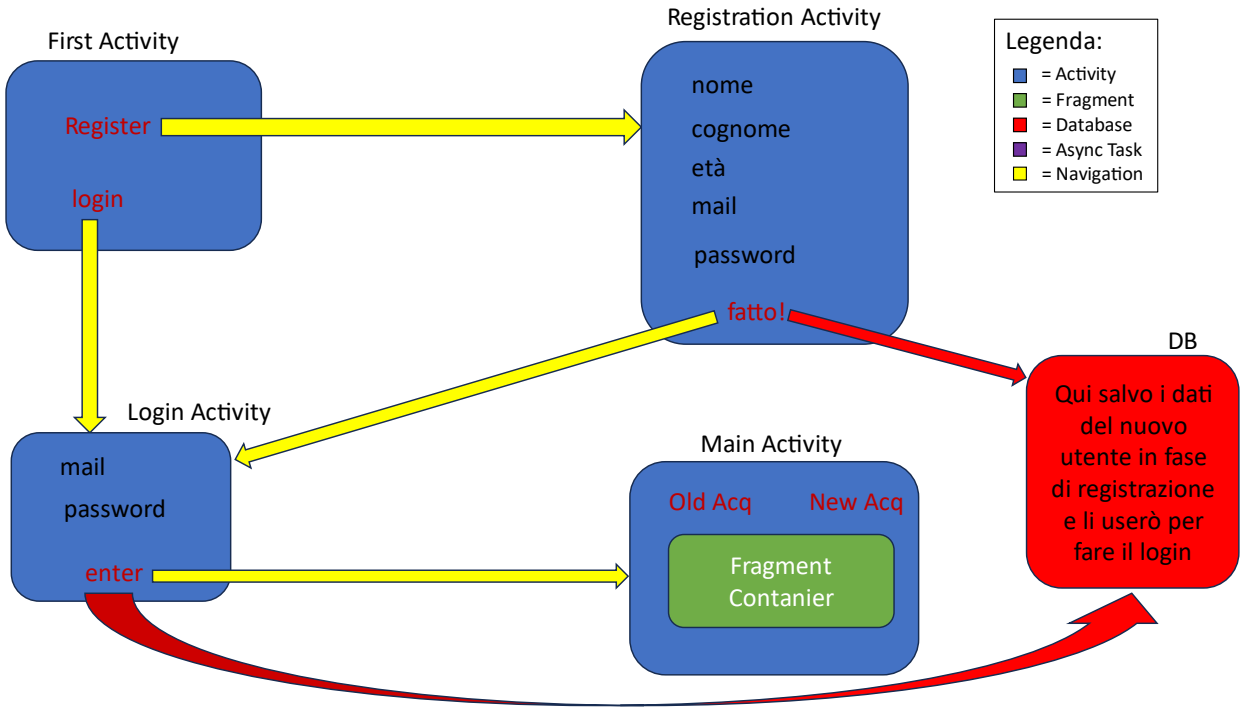


Fig 2: registrazione e login dell'app

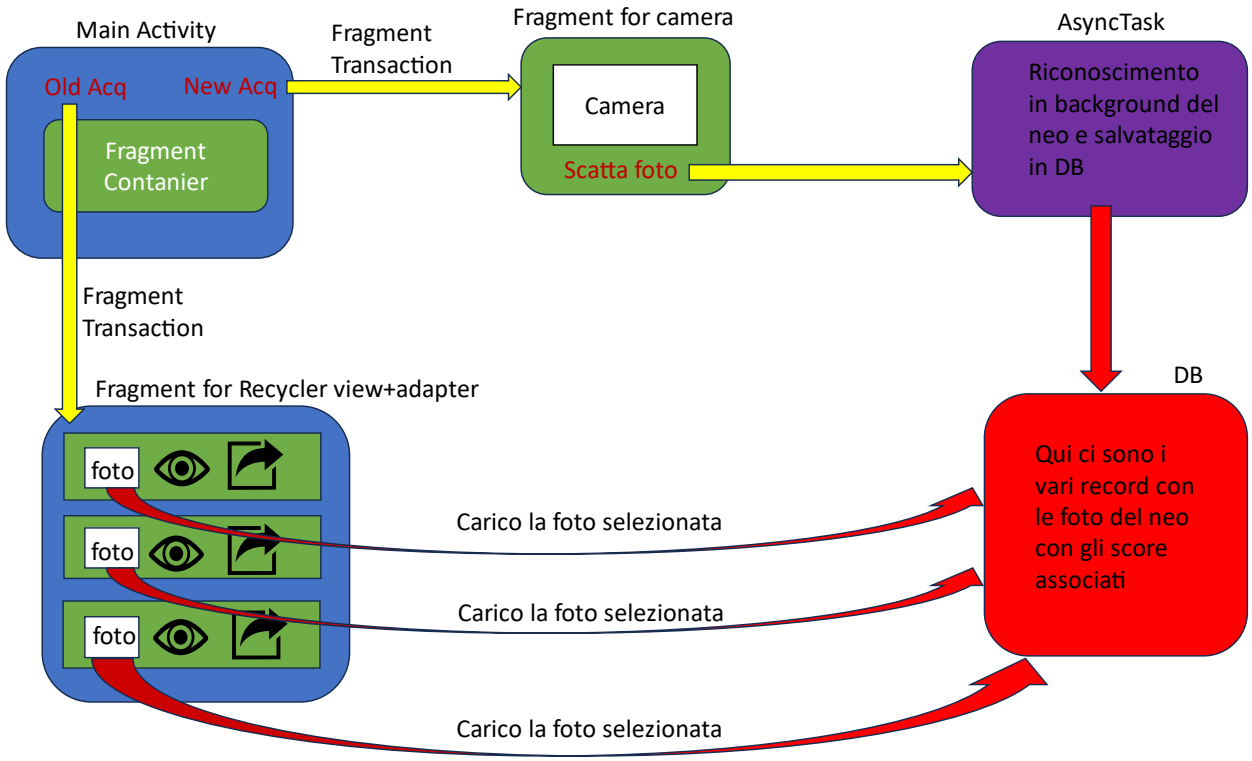


Fig 3: fragment transaction, acquisizione foto e relativa analisi

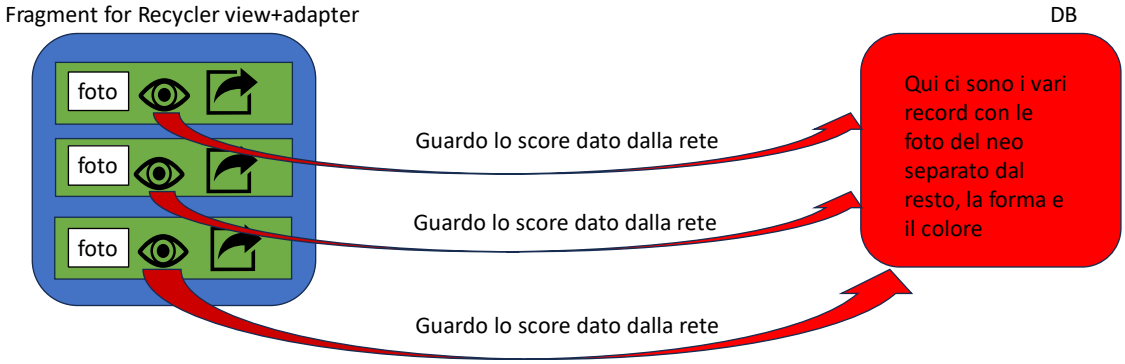


Fig. 4: Visualizzazione score dato dalla rete

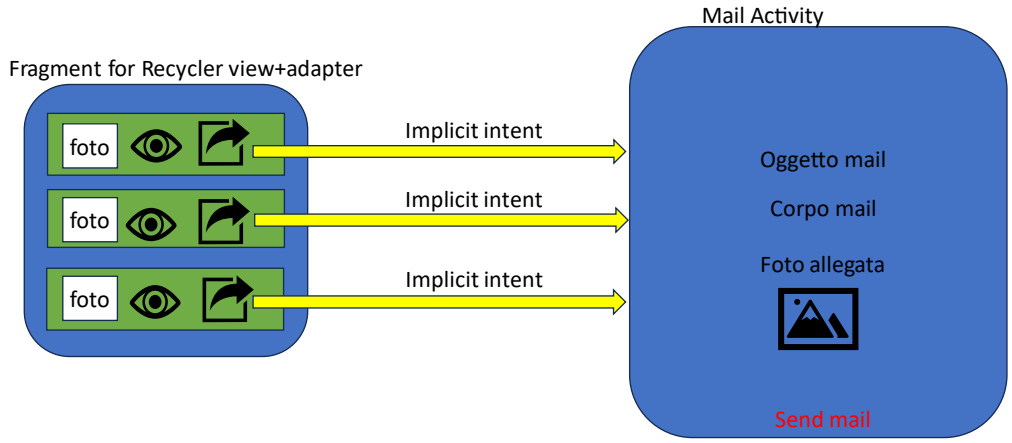


Fig. 5: Invio della foto tramite mail

Database:

durante l'utilizzo dell'app vengono generate molte informazioni di cui è bene tenere traccia in modo "permanente" o, quanto meno, si vuole la possibilità di avere a disposizione tali informazioni quando richiesto.

Per far fronte a tale necessità, ho associato un database all'app. Il database si chiama "NEO_DB" ed è tanto semplice quanto efficace.

Esso è costituito da due sole tabelle:

una per le informazioni dell'utente (T_JAM_UTENTE)

una per le informazioni dell'immagine (T_JAM_IMAGO)

Le due tabelle sono messe in relazione 1: N con la N entrante nella tabella dell'immagine. Ho scelto tale tipo di relazione perché un utente può scattare varie immagini nel corso dell'utilizzo dell'app ma un'immagine una volta scattata, è stata creata da un solo utente.

Tale modellizzazione è conveniente perché la primary key della tabella dell'utente finisce col diventare una foreign key nella tabella dell'immagine e quindi nel momento in cui un utente vorrà vedere le immagini che ha scattato (e solo le sue), basterà utilizzare tale vincolo per far apparire quanto desiderato.

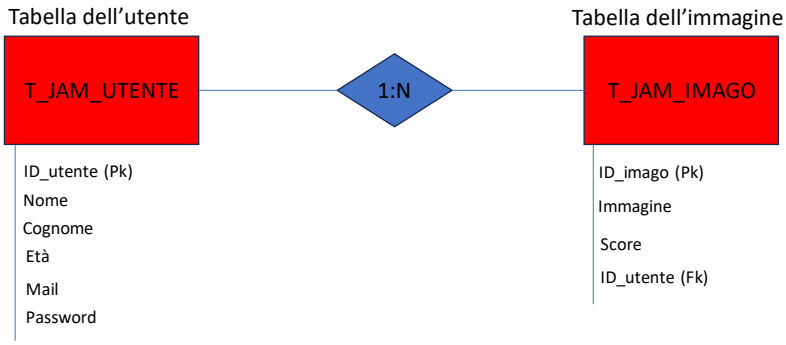


Fig. 6: Diagramma entità associazioni

L'implementazione del database nel software avviene tramite 3 classi:

T_JAM_UTENTE che contiene il nome della tabella dell'utente e i nomi dei relativi campi

T_JAM_IMAGO che contiene il nome della tabella dell'immagine e i nomi dei relativi campi

DB_Helper che è un'estensione della classe SQLiteOpenHelper e serve per creare il Database, le relative tabelle e vari metodi che implementano delle query utili al funzionamento dell'app.

Tra i metodi utili ci sono:

-onCreate (…) che crea le tabelle

-insertUtente (…) che inserisce i dati di un nuovo utente

-duplicati (…) che controlla che la mail inserita non sia già stata utilizzata (una mail può avere un solo account)

-checkloginUtente (…) che serve per il corretto login dell'utente

-insertSalvaImmagine (…) che serve per l’inserimento dell’immagine nel DB una volta fatta la predizione tramite la rete neurale

-prelevaDati (…) che serve per prendere i dati dalla tabella delle immagini e passarli al RecyclerView. È importante notare che tale metodo prende solo i dati relativi all’ ID_utente di chi ha effettuato l’accesso poiché questo campo, nella tabella delle immagini funge da foreign key.

Da qui in poi ci sono dei metodi che non servono per l'app finale ma che ho comunque creato per correggere eventuali errori di percorso:

-selectLook (…) che mostra la lista degli utenti registrati

-updateEta (…) che serve per cambiare l’età inserita in fase di registrazione

-controllaScore (…) che serve per vedere come il DB ha salvato lo score (è un float e volevo vedere se la rappresentazione in virgola mobile poteva dare problemi nel passaggio da DB a un textView, tipo quando invece dell’apostrofo ti appare “&#39”)

Registrazione/Login:

Appena l'utilizzatore apre l'app si trova nella Activity chiamata FirstActivity che è costituita da 2 Button, uno è adibito alla registrazione dell'utente, e uno è adibito al suo accesso (previa registrazione). I 2 bottoni hanno un listener ad essi associato, il bottone di registrazione porta l'utente nella RegistrationActivity mentre il bottone di login lo porta nella LoginActivity.

Nella RegistrationActivity l'utente inserirà il suo nome, cognome, la sua età, la sua mail e una password in dei campi editabili (EditText) e successivamente premerà il bottone in fondo al layout dell'activity che effettuerà l'inserimento dell'utente come record della tabella T_JAM_UTENTE (grazie al metodo insertUtente (…) visto prima) e in seguito lo porterà, tramite un intent esplicito, alla LoginActivity. Se qualcosa dovesse andare storto (campi vuoti, o un'età negativa) apparirà un Toast che segnalerà il problema all'utente e l'inserimento non avverrà.

Nella LoginActivity, l'utente inserisce la mail e la password e premerà un bottone che farà partire i metodi checkloginUtente (…) e duplicati (…) che controlleranno che quell'utente già esista nella tabella T_JAM_UTENTE, che ci sia una sola occorrenza della sua mail nella suddetta tabella e che la mail e password inserite negli EditText della LoginActivity coincidano con quelle nella tabella. Nel caso in cui tutto vada bene l'utente verrà portato nella MainActivity tramite un intent esplicito.

È opportuno far presente che, il campo della password (sia in fase di registrazione che login) dal punto di vista XML è un EditText il cui inputType è "password". Così facendo si ha la visualizzazione ad asterischi di quanto scritto per evitare che la password venga vista da gente "non autorizzata".

Il campo dell'età nella RegistrationActivity dal punto di vista XML è un EDitText il cui inputType è "number" per evitare errati inserimenti (l'età viene scitta a parole invece che a numero), di conseguenza nel momento del suo inserimento cambierà il tipo di tastiera con cui l'utente si deve interfacciare.

MainActivity:

Il layout della MainActivity è costituito da 2 Button e 1 fragment container.

I due Bottoni sono chiamati new_acq (che permette di fare una nuova acquisizione di un'immagine) e old_acq (che fa vedere le vecchie acquisizioni).

Al momento dell'accesso alla main Activity per la prima volta, nel fragment container c'è un fragment che chiede all'utente cosa vuole fare, di seguito l'utente dichiarerà il suo intento tramite i due bottoni. Entrambi i bottoni eseguono, alla loro pressione, una fragment Transaction. Il bottone new_acq mette nel fragment container il fragment chiamato CameraFragment mentre il bottone old_acq mette nel container il fragment chiamato PastImagesFragment.

Durante la fragment transaction viene creato un bundle per passare dei dati dall'activity al fragment che serviranno in seguito.

CameraFragment:

Quando l'utente transisce per la prima volta in questo fragment gli viene chiesto di dare l'autorizzazione all'utilizzo della camera per acquisire l'immagine. Tale operazione viene gestita dall' inserimento nel manifest dei permessi relativi all'uso della camera, vale a dire l'istruzione

```
<uses-permission android:name="android.permission.CAMERA"/>
```

e anche dal paradigma onRequestPermissions che si assicura che il permesso relativo alla camera sia uguale a PERMISSION_GRANTED. Se il permesso non viene dato l'utilizzo di tale fragment viene impedito all'utente, se viene dato allora gli viene consentito l'accesso al layout del fragment

Il layout di CameraFragment è costituito da un ImageView e da un Button chiamato btn_acquire. Il btn_acquire, una volta premuto fa partire un intent implicito relativo all'uso della fotocameraper scattare la foto:

```
Intent intent=new Intent(MediaStore.ACTION_IMAGE_CAPTURE);
startActivityForResult(intent,Camera_code);
```

Ora l'utente può scattare la foto e successivamente viene eseguita la onActivityResult (…) che controlla che il permesso alla camera sia stato dato e che sia stata scattata una foto non nulla. Qualora ciò vada bene, viene creata una bitmap a partire dalla immagine acquisita e viene impostata nell'ImageView del Fragment tramite il metodo setImageBitmap (…)

È opportuno precisare che l'utilizzo della fotocamera e la possibilità di mostrare sullo schermo del telefono quello che coglie dal mondo esterno è possibile grazie alla libreria OpenCV, una libreria open source che permette non solo l'acquisizione di immagini dalla fotocamera ma permette anche anche il processing delle suddette. In particolare, da questa libreria sono stati usati metodi per convertire l'immagine in una matrice e viceversa poichè la rete neurale vuole in input una matrice ma l'ImageView vuole una bitmap quindi serviva uno stratagemma per convertire l'immagine nel formato più opportuno all'occorrenza.

Il metodo onActivityResult (…), dopo aver acquisito l'immagine, crea un processo in background e lo esegue al fine di analizzare l'immagine appena scattata. Tale studio è fatto in background poiché è oneroso in termini di risorse e tempo, quindi si vuole non appesantire il main thread ed evitare il malfunzionamento dell'app.

AsyncImage e ImageClassification:

Per capire il ruolo di queste due classi partiamo dalle seguenti istruzioni:

```
AsyncImage asyncImage=new AsyncImage(img_cam,imageClassification,myDB,infoSave);
asyncImage.execute(img_cam);
```

Nella prima istruzione si crea un'istanza di AsyncImage chiamata asyncImage (che è un'estensione di AsyncTask) e la si inizializza col relativo costruttore da me gestito. Nella seconda istruzione si fa partire il processo in background.

Il costruttore prende in ingresso 4 dati:

-img_cam che è l'ImageView dove viene messa la foto una volta analizzata dalla rete neurale

-myDB che è un'istanza di DB_Helper per salvare la foto nel Database

-imageClassification che è un'istanza della classe ImageClassification che contiene la rete neurale ed effettua la predizione

-infoSave che è l'estensione della classe InfoSave, una classe creata ad hoc per passare all' AsyncTask informazioni utili.

La classe InfoSave contiene 2 soli attributi: uno atto a contenere l'id dell'utente e l'atro per contenere il valore dello score ottenuto dalla rete.

Oltre a ciò ci sono le relative getter e setter per i suddetti attributi.

La necessità di creare tale classe "di supporto" è dovuta al fatto che io devo interagire con tali valori durante un processo in background, ma non posso passare in input al costruttore di un AsyncTask tipi di dati primitivi. Allora ho creato la classe InfoSave (l'ho creata io quindi, ovviamente, non

è un tipo di dato primitivo) per aggirare tale problema. Tale classe mi permette di ottenere l'id dell'utente per effettuare il salvataggio dell'immagine nel Database e mi permette di portare lo score della rete anche fuori dall'AsyncTask una volta che questo ha terminato il suo lavoro.

La classe ImageClassification contiene i seguenti metodi:

-public ImageClassification (AssetManager assetmanager, String modelPath,int inputSize) che è il costruttore di tale classe. Esso prende in input Un manager per gli Asset folder dell'applicazione, il nome della rete neurale e la dimensione del primo layer per creare un interprete che contiene la rete correttamente prelevata dall'Asset folder dove è stata salvata

-private ByteBuffer loadModelFile (AssetManager assetmanager, String modelPath) che è un metodo di supporto usato nel costruttore per prelevare il modello della rete dall'Asset folder e convertirlo in tipo ByteBuffer per una corretta leggibilità del modello

-private ByteBuffer createBitmapToByteBuffer (Bitmap scaledbitmap) che serve per convertire un'istanza di Bitmap in un'istanza di ByteBuffer

-public Mat recognizeImage (Mat mat) che è il metodo più corposo di tutti. Tale metodo prende le dimensioni della matrice passata in input e usa tali valori per fare un crop dell'immagine in modo da prendere il 90% dell'immagine ottenendo quindi una nuova bitmap; tale bitmap viene convertita in ByteBuffer tramite il metodo createBitmapToByteBuffer (…) di cui trattato sopra. Una volta fatto ciò, viene chiamato il metodo run (…) sull'interprete che avevo già creato nel costruttore di ImageClassification (…). Tale metodo prende in input2 dati: il ByteBuffer che abbiamo ottenuto tramite tramite il metodo createBitmapToByteBuffer (…) e "output" che è un Float che conterrà il punteggio associato all'immagine. col metodo run la rete neurale studia l'immagine e le associa un punteggio che verrà poi confrontato con una soglia e a seconda di ciò farà apparire sull' immagine una scritta verde ("neo benigno") o rossa ("neo maligno") a indicare il valore della predizione. Tale metodo quindi

resituisce la matrice contente l'immagine scattata con l'aggiunta della scritta "neo/benigno/maligno".

Una volta che l'istanza della classe ImageClassification ha concluso il suo lavoro (restituendo la matrice con la predizione), il metodo doInBackground dell'istanza di AsyncImage salva il valore della predizione nell'attributo score della istanza della classe InfoSave che avevamo precedentemente creato.

Una volta eseguite queste istruzioni, il metodo onPostExecute inizia il suo lavoro. Tale metodo prende in input, ovviamente, la matrice con la predizione in uscita dalla doInBackground e la converte in bitmap così da mostrarla nell'imageView di CameraFragment. È possibile fare ciò perché nel costruttore della classe AsyncImage avevo messo una weakReference a tale imageView. In questo modo l'utente ha visione "immediata" della predizione fatta dalla rete neurale.

Nel metodo onPostExecute avviene anche un'altra cosa, la bitmap precedentemente ottenuta viene convertita in Array di byte e successivamente salvata nel Database (nella T_JAM_IMAGO) tramite il metodo insertSalvaImmagine che prende in input l'immagina da salvare (di tipo Array di byte), l'id dell'utente che funge da foreign_key (che abbiamo a disposizione poiché nella creazione dell'istanza di AsyncImage avevamo passato un'istanza di InfoSave al costruttore), e lo score (che abbiamo anch'esso a disposizione grazie all'istanza della classe InfoSave).

È opportuno evidenziare che durante la conversione della bitmap in Array di byte viene utilizzato il metodo compress (…) e tra i parametri in input di tale metodo ho imposto che l'immagine avesse un formato di compressione PNG. Ho preferito il formato PNG rispetto al JPEG perché PNG è un formato di compressione senza perdite e, visto che tale immagine verrà poi mandata al medico tramite mail, conviene che l'immagine abbia meno "modifiche" possibili al proprio contenuto (vale a dire al contenuto dei propri pixel).

Rete Neurale:

la rete neurale è stata creata e allenata grazie a kaggle.com che mette a disposizione sia dataset di vario tipo che dei processori CPU e GPU per allenare la rete in tempi "umani". Come linguaggio di programmazione per creare la rete ho scelto Python perché è Open Source e perché è dotato di varie librerie di supporto per la progettazione di reti neurali (Tensorflow, Keras e i loro vari sottomoduli).

Il dataset (qui: https://doi.org/10.34970/2020-ds01) per allenare la rete è stato fornito dalla International Skin Imaging Collaboration (ISIC) ed è stato messo a disposizione per la Society for Imaging Informatics in Medicine (SIIM) che ha organizzato una competizione dove i partecipanti dovevano creare la miglior rete che identificasse correttamente i nei benigni e maligni.

Io ho usato, per allenare la mia rete, lo stesso dataset ma con le immagini "riscalate" a dimensione 256x256.

Grazie a kaggle.com ho ottenuto la rete addestrata, ma in formato .h5 . Per usare una rete neurale su un telefono (che ha meno risorse di un PC) è opportuno (se non obbligatorio) convertirla in formato .tflite . Tale operazione non l'ho fatta su kaggle.com ma sull' editor Spyder di Anaconda da mio PC. Non nego che sia possibile fare tale operazione anche su kaggle.com ma io non ci sono riuscito e ho deciso di fare da me, perché arrivato a tele punto del progetto non avevo più bisogno di dataset e GPU potenti, ma solo di convertire un file di un certo tipo in un altro.

Una volta convertita la rete in formato .tflite, ho creato un asset folder e ci ho messo dentro la rete; dopodiché ho aggiunto le librerie di TensorFlow nelle dependencies del gradle (cercando le versioni più adatte al mio environment).

PastImagesFragment:

Lo scopo di tale fragment è la visualizzazione delle immagini, una volta che sono state analizzate, e la spedizione delle suddette tramite email a medico.

La prima cosa che fa tale fragment è il prelevare i dati da visualizzare dal DataBase tramite il metodo public void prelvatore_DIDati (DB_Helper mydb). Tale metodo seleziona dalla T_Jam_Imago, l'immagine, lo score e la primary_key e li usa per creare un dato di tipo PlaceHolderItem (tramite il suo relativo costruttore) per poi aggiungerlo come elemento dell'Array di PlaceHolderItem che è chiamato ITEMS.

Una volta, eseguito tale metodo, il fragment crea un RecyclerView e nel metodo setAdapter ci passa ITEMS cosicché l'Adapter sappia quali sono i dati da mostrare.

L'utente così vedrà le varie righe del RecyclerView, ognuna delle quali mostrerà a sinistra l'immagine analizzata mentre a destra, messi in colonna, la primary_key dell'immagine, un'icona, e lo score associato all'immagine.

Di questi elementi, l'icona è cliccabile e permette di mandare l'immagine relativa a quel record al medico tramite un intent implicito di tipo ACTION_SEND.

Ciò è possibile perché, in fase di progettazione, quando ho esteso la classe RecyclerView.Adapter<…> tra le parentesi angolari dovevo passargli anche il ViewHolder di cui devo creare il costruttore ed è proprio nel costruttore che ho associato i vari elementi del layout di una riga generica del RecycklerView ai relativi elementi Java e, inoltre, ho chiamato il metodo setOnClickListener (…) sull' icona. In tale metodo, mi salvo il numero del record su cui sono grazie al metodo getAbsoluteAdapterPosition (…), da ciò ricavo la relativa immagine e quindi chiamo il metodo onIconClick (…) dell'interfaccia AvvisaMedico che ho creato (dove dentro c'è solo la definizione del metodo onIconClick).

Nel metodo public void onIconClick (int pos, Bitmap bitmap) salvo l'immagine in un file nella memoria interna, ottengo l'URI di tale file grazie al file Provider che ho inserito nel manifest, e infine creo l'intent implicito di tipo ACTION_SEND specificando che sto mandando un immagine grazie al metodo .setType ("image/*") e specificando l'oggetto della mail, il testo della mail e quale allegato mandare (l'URI del file relativo all'immagine in questione).

È importante notare che l'utilizzo dell'interfaccia è stato possibile perché l'ho mandata in input al costruttore del ViewHolder e alla sua onCreateViewHolder (…).

Ovviamente l'oggetto della mail e il testo sono editabili, ho aggiunto del testo di default solo per evitare situazioni di errore dovuti ai sudetti campi vuoti.